

MASTER OF COMPUTER SCIENCE

SEC101: SCIENTIFIC WRITING AND COMPUTATIONAL ANALYSIS

TOOLS [0-0-2]

Course Code	Credits	Credit distribution of the course			Pre-requisite of the course (if any)
		Lecture	Tutorial	Practical/Practice	
SEC101	2	0	0	2	NIL

Course Objective:

The objective of this course is to develop proficiency in the use of software tools required for project development.

Course Learning Outcomes:

Upon successful completion of this course, students will be able to

1. create a basic LaTeX document, format text with different styles, and apply fundamental document structures such as sections, paragraphs, and formatting options.
2. create and manipulate matrices and arrays in MATLAB, understanding their importance in numerical computation and data analysis.
3. apply optimisation techniques and perform curve fitting tasks in MATLAB, including fitting data to a model and minimizing error.

Syllabus:

Unit-I

LaTeX : Basic Document Setup, and Formatting Text, Lists and Tables, Multi-Column Layouts, Mathematical Equations, Figures and Graphics, Customizing Fonts and Colors, References, Citations, and Headers/Footers/Page Numbering, Creating a Resume or CV.

Unit-II

MATLAB: Basic MATLAB Operations, Matrices and Arrays, Plotting and Visualization, Loops and Conditional Statements, Functions and Scripts, File Handling, Signal Processing, Image Processing, Optimization and Curve Fitting, Simulink Basics

Readings:

1. <https://guides.nyu.edu/LaTeX/sample-document>
2. Brian R. Hunt, Ronald L. Lipsman, Jonathan M. Rosenberg. *A Guide to MATLAB: For Beginners and Experienced Users*, Cambridge University Press, 2017.

List of Practical's

1. Basic Document Setup and Formatting Text: Create a new document with a specific page size, orientation, and margins, Format a paragraph using different font styles (bold, italic, underline) and sizes, Apply alignment options (left, right, center, justified) to different text blocks, Use styles to format headings and body text consistently. **(4 hours)**
2. Lists and Tables, Multi-Column Layouts: Create a bulleted and numbered list for an agenda or shopping list, design a table to represent student grades or inventory details, merge and split cells in the table, and apply borders and shading. Create a newsletter-style document using two or three-column layouts. **(4 hours)**
3. Mathematical Equations: Insert and format basic math expressions, such as fractions, superscripts, and subscripts. Write complex equations, like the quadratic formula, matrices, and integrals, using equation tools or LaTeX syntax (if applicable), and align multiple equations properly using equation editors or tab alignment. **(4 hours)**
4. Figures and Graphics, Customising Fonts and Colours: Insert an image or figure into the document with a caption, Resize and position the image using wrap text and alignment options, Change font family and colour scheme for different document elements, Create a cover page with custom fonts, colours, and images. **(4 hours)**
5. References, Citations, and Headers/Footers/Page Numbering: Insert a bibliography and add citations using a reference manager or built-in citation tools, Apply consistent header and footer designs across the document, Insert and format page numbers (e.g., Roman numerals for intro, Arabic for content), Create a Table of Contents and update it automatically. **(4 hours)**
6. Creating a Resume or CV: Use a template or design a CV layout from scratch with appropriate sections (Education, Experience, Skills), Insert a profile photo, format contact details, and add social media links, Apply proper use of whitespace, bullets, and alignment to ensure clarity and readability, Export the CV as a PDF and check formatting consistency. **(4 hours)**

7. Introduction to MATLAB Interface and Basic Commands: Using command window, editor, and workspace, Arithmetic operations, using help, clc, clear, who, whos. Variable Assignment and Data Types: Creating scalars, vectors, complex numbers, Type conversion, and precision handling, Matrix Creation and Manipulation: Defining matrices, transposition, reshaping, Indexing, slicing, concatenation, Matrix multiplication, inversion, determinant, eigenvalues. **(4 hours)**
8. Plotting and Visualization 2D and 3D Plotting: *plot()*, *subplot()*, *title()*, *xlabel()*, *ylabel()*, *legend()*, 3D plots: *mesh()*, *surf()*, *contour()*. Data Visualisation with Customisation: Bar graphs, pie charts, histograms, Line styles, markers, colour changes. **(4 hours)**
9. Loops and Conditional Statements: Using for, while, and if-else, writing loops to sum a series, the Fibonacci series, and Conditionals to check prime numbers or grading systems, Functions and Scripts, Creating User-Defined Functions, writing a function to compute factorial, and standard deviation. **(4 hours)**
10. Using Scripts for Automation: Writing scripts to read input, process, and display output, File Handling: Reading and Writing Files, reading data from .txt, .csv using *fopen*, *fscanf*, *textscan*, *readmatrix()*. Writing output to files. **(4 hours)**
11. Signal Processing: Basic Signal Generation and Analysis, generating sine, square, and triangular waves, plotting signals, and performing FFT. Filtering Signals: Applying FIR and IIR filters, using *filter()*, *butter()*, and *freqz()*. **(4 hours)**
12. Image Processing: Image Reading and Display, Read and display an image using *imread()*, *imshow()*, and *rgb2gray()*. Image Enhancement and Filtering: Histogram equalization, edge detection, smoothing. **(8 hours)**
13. Optimisation and Curve Fitting: Curve Fitting using *polyfit()* and *fit()*. Fit a polynomial or custom function to data, Evaluate goodness of fit, optimisation using *fminsearch*, *fmincon*: Minimise a nonlinear function with constraints. **(8 hours)**

SEC201: Ethics for Responsible AI [1-1-0]

Course title	Credits	Credit distribution of the course			Pre-requisite of the course (if any)
		Lecture	Tutorial	Practical/Practi ce	

Ethics for Responsible AI	2	1	0	1	Knowledge of AI and ML concepts
----------------------------------	----------	----------	----------	----------	--

Course Objectives:

The objective of this course is to equip students with foundational concepts of ethics and relate them to the ethical concerns in the rapidly advancing field of Artificial Intelligence. The course also imparts practical skills to the students to address ethical challenges in AI development and deployment through important case studies. Taught using recent research publications, the course aims to develop competence to adapt to new ethical challenges that arise due to rapid advancements in AI.

Course Learning Outcomes:

At the end of this course, students will be able to

1. explain the fundamental theories of ethics.
2. relate the theories to the ethical concerns regarding Artificial Intelligence.
3. understand the practical implications of the ethical concerns.
4. apply mitigation strategies using available tools.
5. equipped to understand and handle new ethical challenges that arise due to swiftly advancing AI technology.

Syllabus

Unit-I

(10 hours)

Foundations of Ethics & Ethical AI: Introduction to Ethics; Ethical theories - Utilitarianism, Deontology, Virtue Ethics; Case Study - The Trolley Problem in autonomous vehicles; AI-Specific Ethical Challenges - Bias and Fairness, Transparency and Explainability, Accountability, Privacy. Case studies for Ethical concerns and Mitigation: Bias and Fairness, Transparency and Explainability, Accountability, Privacy (hiring, criminal justice, healthcare, autonomous driving etc.), Technical challenges for addressing ethical concerns; Mitigation strategies and techniques - Federated learning, differential privacy, Algorithmic manipulation, Data Curation; Explainability tools (SHAP and LIME).

Unit-II

(5 hours)

AI Governance: Societal risks - Job displacement, misinformation, environmental costs, deepfake; IEEE guidelines; ACM Code of Ethics for development and deployment of AI; Introduction to "right to explanation" in EU's GDPR, EU AI Act, Indian scenario; Future challenges - AGI Ethics, autonomous weapons.

Reading:

1. Jobin, A., Ienca, M. & Vayena, E. (2019). *The global landscape of AI ethics guidelines*. Nat Mach Intell 1, 389–399, 2019.
2. Radanliev, P., Santos, O., Brandon-Jones, A., & Joinson, A., *Ethics and responsible AI deployment*. Frontiers in Artificial Intelligence, 7, 1377011, 2024.
3. Pant, A., Hoda, R., Spiegler, S. V., Tantithamthavorn, C., & Turhan, B., *Ethics in the age of AI: An analysis of AI practitioners' awareness and challenges*. ACM Transactions on Software Engineering and Methodology, 33(3), 1-35, 2024
4. Willem, T., Fritzsche, M. C., Zimmermann, B. M., Sierawska, A., Breuer, S., Braun, M., ... & Buyx, A., *Embedded Ethics in Practice: A Toolbox for Integrating the Analysis of Ethical and Social Issues into Healthcare AI Research*. Science and Engineering Ethics, 31(1), 1-22, 2025.

List of Practicals

1. AI Ethics Case Study Debate (<https://www.aiethicist.org/ethics-cases-registries>)

Choose an AI ethics case study from the above mentioned URL (e.g., facial recognition in policing, autonomous vehicle accidents, ChatGPT and misinformation). Study the selected case in depth and work in a team of six students.

Each team will role-play the following three stakeholder groups: Developers, Victims of bias and Regulators (Two students per stakeholder group). Prepare 5-minute arguments both in favour of and against the deployment of the AI system under discussion. Each team must present a balanced and well-researched perspective, considering ethical, technical, and societal implications. Next, prepare a one-page individual write-up that should reflect on one of the debates conducted in class, in addition to the one in which they participated. (14 hours)

2. Select and download a classification dataset of your choice. Train a complex model, such as a Random Forest or a Neural Network, on the dataset. Use LIME to explain the model's predictions for three correctly classified and three misclassified test instances.

Submit the notebook containing the complete code for data preprocessing, model training, evaluation, and LIME-based explanation. Also, submit a report summarising your findings, including visualisations from LIME and your interpretation of the explanations. (8 hours)

3. Choose one of the following datasets.

- MBIC – A Media Bias Annotation Dataset: <https://arxiv.org/pdf/2105.11910>
- COMPAS - Recidivism Racial Bias: <https://www.kaggle.com/danofer/compass>
- Adult Income dataset: <https://archive.ics.uci.edu/dataset/2/adult>
- Dataset provided in the class

Train a classification model (e.g., Decision Tree, Logistic Regression, or XGBoost) on a suitable dataset. Use SHAP (SHapley Additive exPlanations) to interpret the model's predictions. Select five correct and five incorrect predictions and analyse them using SHAP values to understand the key features contributing to each decision. Interpret feature importance and identify patterns that influence the model's decisions. Detect any potential biases or discriminatory behaviour in the model's predictions. Summarise your findings, and reflect on how the identified biases can be mitigated to make the model fairer and more robust. (8 hours)

MASTER OF COMPUTER APPLICATIONS (MCA)

SEC101: SOFTWARE TOOLS AND TECHNIQUES [0-0-2]

Course Code	Credits	Credit distribution of the course			Pre-requisite of the course (if any)
		Lecture	Tutorial	Practical/Practice	
SEC101	2	0	0	2	

Course Objective:

The objective of this course is to develop proficiency in the use of software tools required for project development.

Course Learning Outcomes:

Upon successful completion of this course, students will

1. have knowledge of the fundamental shell tools and scripting techniques used in Linux or Unix-like environments.
2. understand the process of booting systems, using live USBs, and understanding their role in system recovery and troubleshooting.
3. apply Markdown for documentation, including its syntax and its applications in writing clear, readable documents.
4. Use Git for version control, including branching, merging, and resolving conflicts in collaborative software development

Syllabus:

Unit I

Foundations of Shell and Command-line Environment: Shell Tools and Scripting, Editors (Vim), Command-line Environment, Debugging and Profiling, Common command-line flags/patterns, Booting + Live USBs, Metaprogramming: Working with Daemons, FUSE, Markdown

Unit II

Advanced Tools and Applications: Version Control (Git), Markdown language, Backup strategies: full, incremental, and differential, Vagrant and VMs, Docker and Containerization, Cloud Providers Overview, manage public and private clouds using OpenStack

Readings:

1. Newham C., *Learning the bash shell: Unix shell programming*. O'Reilly Media, Inc., 2005
2. Shotts W. *The Linux command line: a complete introduction*. No Starch Press. 2019.
3. <https://git-scm.com/book/en/v2>
4. <https://www.techtarget.com/searchdatabackup/feature/Full-incremental-or-differential-How-to-choose-the-correct-backup-type>
5. <https://developer.hashicorp.com/vagrant/docs>
6. <https://docs.docker.com/get-started/>
7. <https://docs.openstack.org/2025.1/>

List of Practical's

1. Shell Tools and Scripting: Write a script to display system information (uptime, disk usage, memory), use of variables, conditionals, and loops, Text Processing with Shell: Use grep, awk, sed, cut, sort, uniq, wc to process log files, count failed SSH login attempts from `/var/log/auth.log`. **(4 hours)**
2. Vim Editor Mastery: Open a file, insert text, delete lines, use undo/redo, use search/replace, split windows, macros, and customise. vimrc. Command-line Environment: Bash Configuration and Aliases, Create and modify `.bashrc` or `.Zshrc` with aliases and functions, set environment variables, use history, jobs, fg, bg, kill, & nohup. **(4 hours)**
3. Shell Script Debugging: Use bash -x and set -x to trace script execution, insert logging and echo for runtime info, Measure script runtime with time, perf, strace, and top, Common Command-line Flags/Patterns, explore tools like find, xargs, tee, tr, yes, watch, cut, paste, and pipeline, Batch rename files or extract columns from CSV using shell commands. **(4 hours)**
4. Creating and Using a Live USB: Create a bootable USB using dd, balenaEtcher, or Rufus, Boot from USB and explore the Linux live environment, Mount and access partitions, Exploring Boot Process, use dmesg, journalctl, systemctl, and lsblk to inspect boot logs and services. **(4 hours)**
5. Metaprogramming: Working with Daemons, FUSE, convert a script into a background daemon process, manage it using systemctl or cron, Install FUSE, mount a simple

FUSE filesystem (e.g., using `hello.py` or `passthrough_fuse`), Create a basic virtual FS using Python or C with `fusepy`. **(2 hours)**

6. Writing and Rendering Markdown: create a `.md` file with headings, lists, code blocks, links, and images. Convert Markdown to HTML/PDF using `pandoc` or preview with VSCode/Typora. Optional: Use `grip` or `markdown-cli` for local web preview. **(2 hours)**
7. Git Basics: Initialize a Git repository, Track files using `git add`, `git commit`, and `git status`, Branching and Merging, Create, switch, and merge branches, Solve a basic merge conflict. **(4 hours)**
8. Remote Repository and Collaboration, push to GitHub or GitLab, Clone, pull, and contribute using forks and pull requests. **(2 hours)**
9. Understand and implement Full and Incremental Backup strategies. Use `tar` or `cp` to create a full backup of the directory. **(2 hours)**
10. Schedule incremental backups using `rsync` or `borg`. Demonstrate how changes from the last full backup are saved using tools like `rdiff-backup`. **(2 hours)**
11. Vagrant and Virtual Machines (VMs): write a Vagrantfile to launch a Linux VM using VirtualBox and automate software installation using shell scripts or Ansible within the Vagrant VM. **(2 hours)**
12. Docker and Containerization: Install Docker on your system and verify the installation using the command `docker info`. Use `docker pull` to download official images of `nginx` and `Ubuntu`. Start a container and explore the `nginx` container by accessing it via the browser. Stop the container **(2 hours)**.
13. In continuation of Lab exercise 11, write a Dockerfile to containerise a simple web app (e.g., Python Flask or Node.js) specifying dependencies and configurations. Build the Docker image and run the container. **(2 hours)**
14. Install *Docker Compose*. Write a simple `docker-compose.yml` file that defines two services: A web app (use Web App created in Lab exercise 12) and A database (e.g., MySQL or PostgreSQL). Demonstrate the Start and stop of both the web and database services. Further, show how Docker can be used to run multiple instances of the web app. **(4 hours)**
15. Explore the three leading cloud platforms: Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP). Further, make a document containing a comparison table that contrasts the main features and services provided by these three cloud service providers. Gather details on the following categories: Compute Services, Storage Services, Networking Services, Machine Learning and Analytics, and Security Services. **(4 hours)**

- 16.** Deploying a Virtual Machine on AWS by creating an EC2 Instance on AWS. Select the right AMI (Amazon Machine Image), instance type, and configure security groups. Further, launch and connect to the EC2 instance created. **(2 hours)**
- 17.** Deploying a Virtual Machine on Azure. Further, deploy a VM on Google Cloud Platform (GCP) using Compute Engine. Further explore the Google Cloud Console and basic VM settings (region, machine type, firewall rules). **(4 hours) (Optional)**
- 18.** Networking Setup: Create Virtual Private Networks (VPCs) in AWS, Virtual Network in Azure, and VPC in GCP. Configure subnets and firewall rules for VMs to communicate securely. **(2 hours) (Optional)**
- 19.** Monitoring and Management: Introduce cloud-native monitoring tools for VM performance (e.g., CPU usage, network traffic). Set up basic alerts and logs for VM health and performance monitoring. **(2 hours)**
- 20.** Cloud providers offer a wide variety of instance types optimised for different workloads (e.g., compute-optimised, memory-optimised, storage-optimised). Discuss AWS EC2 instance families such as T-series and M-series. Similarly, explore Azure's virtual machine types (e.g., A-series, D-series, E-Series). Review GCP machine types (e.g., N1-standard, C2, and M-series). Compare the trade-offs between choosing a general-purpose instance and specialised instances for specific workloads. **(6 hours)**
- 21.** Build and Manage Public and Private Clouds using OpenStack: Install OpenStack using DevStack or Packstack on a VM or cloud host, launch instances, create networks, and assign floating IPs using Horizon (web UI) or CLI, write a script to automate instance creation using openstack CLI or heat templates. **(2 hours)**
- 22.** Backup Containers and VM Snapshots: Demonstrate backing up Docker volumes and VM snapshots. Use Git + Docker + Vagrant + Cloud deployment for a CI/CD mini project. **(4 hours)**

